

Assignment 6 - Flat World Climate

Rasmus Madsen, Asger Andersen, Benjamin Siddique

January 2026

1 MPI parallelise the program

In our code we implemented a 2D domain decomposition. For doing checksum and other validations checks we used MPI.Allreduce to gather local checksums from each of the ranks. We used MPI.Gatherv when collecting all the local grid into a global one, such that we could write to the disc. The domain decomposition is done by segmenting the global world into smaller worlds. The iterations to find these sub worlds are done by finding the minimum circumference of a sub world, while still keeping the grid boundaries integers. The 0'th rank acts as the distributor and starts by broadcasting the global albedo data. The other ranks then extract the data of their own sub world from the global world data. After this point, the 0'th rank now also acts as a compute core and only gather if it needs to write to disc.

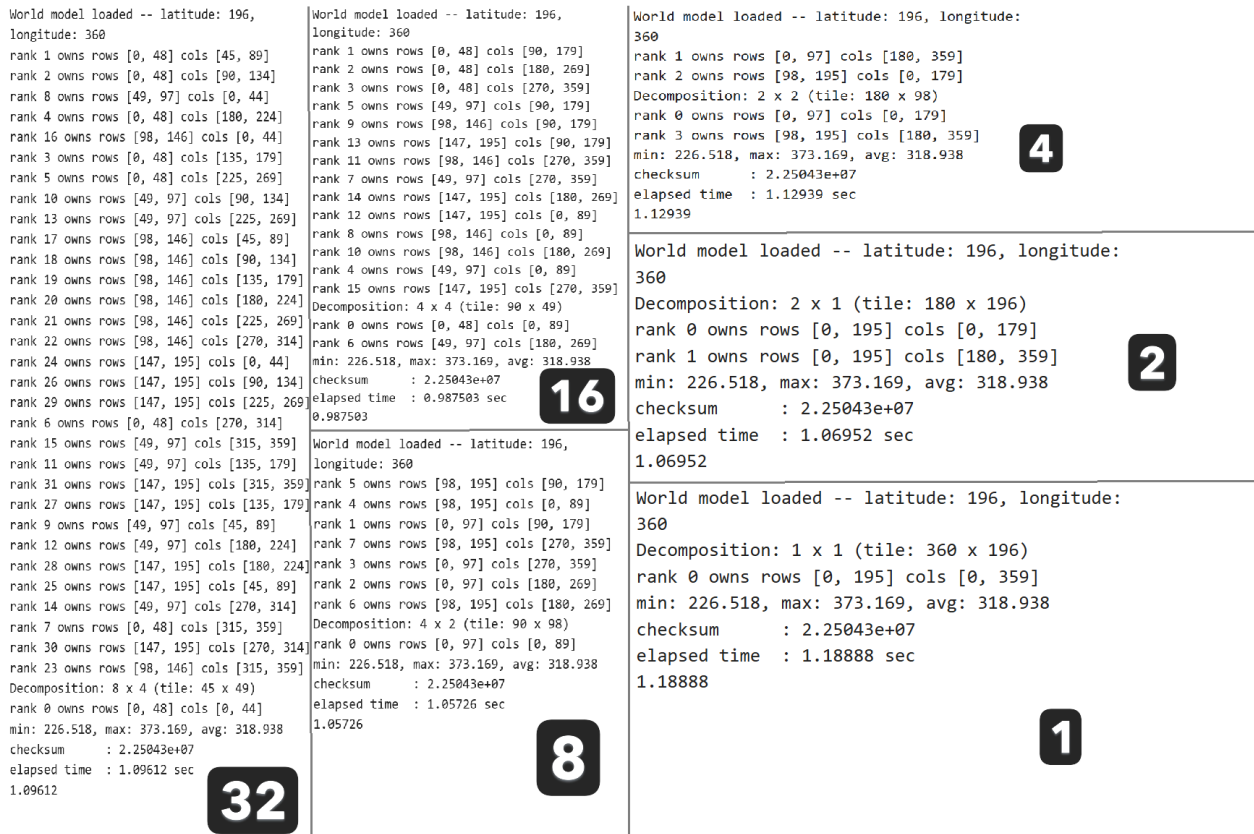


Figure 1: Prove that the checksums are all the same. The order are from 32 to 1 cores (first down then to the right).

2 Strong scaling SLURM

For benchmarking the strong scaling we used 1, 2, 4, 8, 16, 32, 64, 128, 256 and 448 cores. For small we used 10,000 iteration, medium 1000 and for large we used 50 iterations. Each run we repeated 5 times. We noticed that there were bad nodes, which gave a extremely long runtime which we excluded from our benchmarking, i. e. why we have used 448 instead of 512 cores, may n001 forever rest in piece.

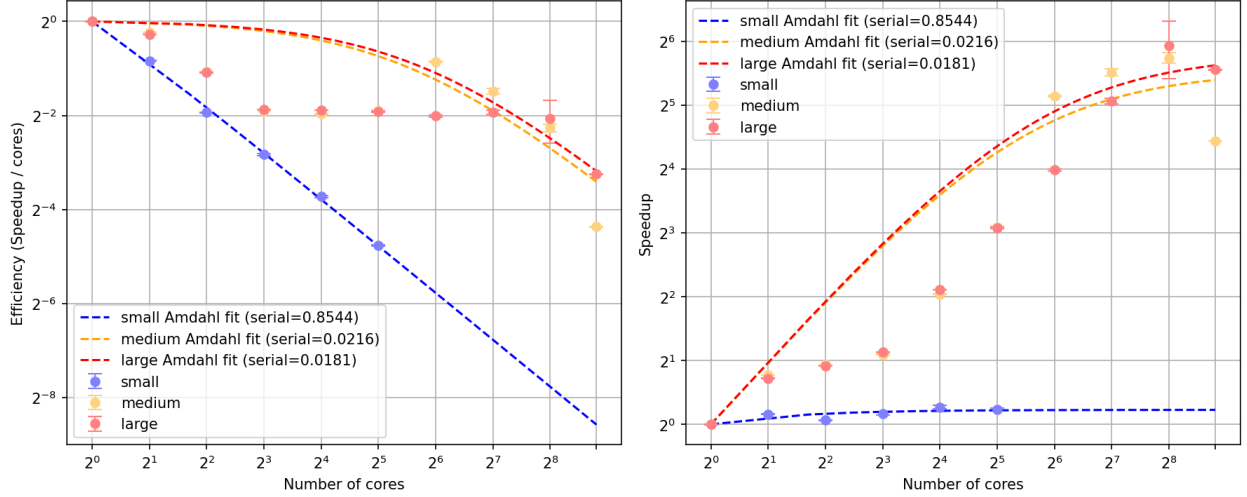


Figure 2: Efficiency and speedup plotted against the amount of cores for constant workload.

Looking at figure 2 its clear that amdahls law fits extremely well to the map with the small resolution. And as expected due to the small size most of the time is spend on overhead, which is the reason that 85% of the time is spend doing serialized task. The medium and large resolution efficiency and speedup follow each other closely. We see that for 2, 4 and 8 cores we get a plateau in the speedup. This is quite weird behavior, and we are not quite sure what causes this plateau as the speedup again rises at 16 cores. This causes the fit to be quite bad and therefore we opt to fit for cores above 16 cores, but still including these weird data points we get a $\approx 2\%$ serial fraction for both the medium and large resolution.

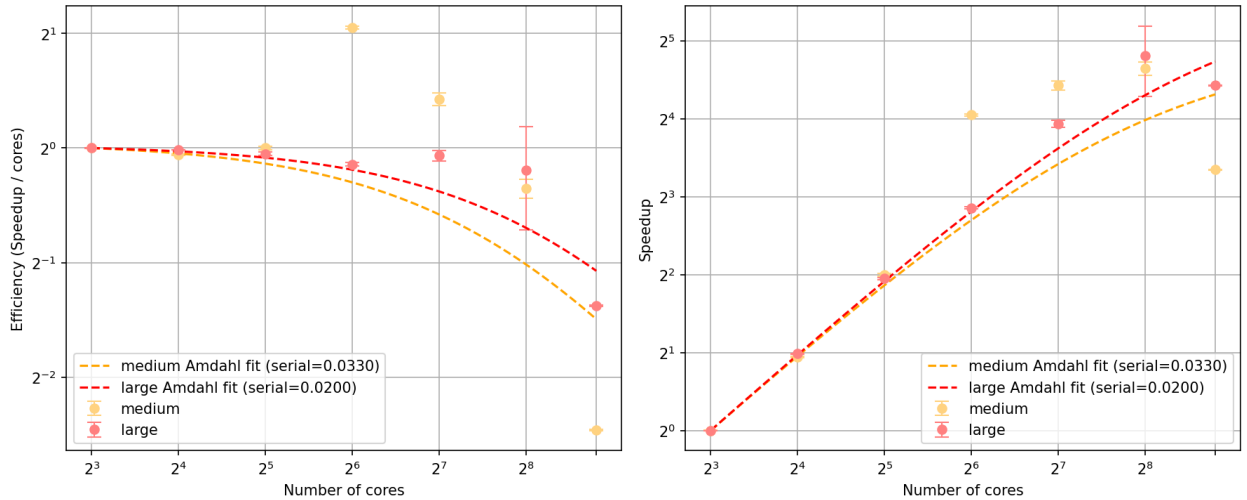


Figure 3: Efficiency and speedup plotted against the amount of cores for the medium and large with the first 3 data points excluded. The workload constant.

Now, the fit reassembles the data quiet a bit better for the large size, but the efficiency drastically drops

when using 448 cores. This is due to how the cluster is set up so communication between numa distances are slower than the communication internally within. When running 7 nodes we are forced to use nodes across numa distances and thus increase communication time between ranks.

3 Appendix — Code

```
#include <vector>
#include <iostream>
#include <H5Cpp.h>
#include <chrono>
#include <cmath>
#include <numeric>
#include <mpi.h>

// Get the number of processes
int mpi_size;

// Get the rank of the process
int mpi_rank;

// 2D decomposition dimensions
int mpi_size_x = 1;
int mpi_size_y = 1;

/** Compute the tile size for a given rank index along one dimension.
 * The first (global_dim % num_parts) ranks get one extra element.
 */
inline uint64_t tile_size_for(uint64_t global_dim, int num_parts, int part_idx) {
    uint64_t base = global_dim / num_parts;
    uint64_t remainder = global_dim % num_parts;
    return base + (part_idx < (int)remainder ? 1 : 0);
}

/** Compute the global offset for a given rank index along one dimension. */
inline uint64_t tile_offset_for(uint64_t global_dim, int num_parts, int part_idx) {
    uint64_t base = global_dim / num_parts;
    uint64_t remainder = global_dim % num_parts;
    return base * part_idx + std::min((uint64_t)part_idx, remainder);
}

/** Representation of a flat world */
class World {
public:
    // The current world time of the world
    double time;
    // The size of the world in the latitude dimension and the global size
    uint64_t latitude, global_latitude;
    // The size of the world in the longitude dimension
    uint64_t longitude, global_longitude;
    // The offset for this rank in the latitude dimension
    long int offset_latitude;
    // The offset for this rank in the longitude dimension
```

```

long int offset_longitude;
// The temperature of each coordinate of the world.
// NB: it is up to the calculation to interpret this vector in two dimension.
std::vector<double> data;
// The measure of the diffuse reflection of solar radiation at each world coordinate.
// See: <https://en.wikipedia.org/wiki/Albedo>
// NB: this vector has the same length as 'data' and must be interpreted in two dimension.
std::vector<double> albedo_data;

/** Create a new flat world.
 *
 * @param latitude    The size of the world in the latitude dimension.
 * @param longitude    The size of the world in the longitude dimension.
 * @param temperature The initial temperature (the whole world starts with the same temperature).
 * @param albedo_data The measure of the diffuse reflection of solar radiation at each world coordinate.
 *                    This vector must have the size: 'latitude * longitude'.
 */
World(uint64_t latitude, uint64_t longitude, double temperature,
      std::vector<double> albedo_data) : latitude(latitude), longitude(longitude),
                                       data(latitude * longitude, temperature),
                                       albedo_data(std::move(albedo_data)) {}

};

double checksum(World &world) {
    double local_cs = 0;
    // only loop *inside* data region — not in ghostzones!
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
        for (uint64_t j = 1; j < world.longitude - 1; ++j) {
            local_cs += world.data[i*world.longitude + j];
        }
    double global_cs = 0;
    MPI_Allreduce(&local_cs, &global_cs, 1, MPLDOUBLE, MPLSUM, MPLCOMM_WORLD);
    return global_cs;
}

void stat(World &world) {
    double local_min = 1e99;
    double local_max = 0;
    double local_sum = 0;
    for (uint64_t i = 1; i < world.latitude - 1; ++i)
        for (uint64_t j = 1; j < world.longitude - 1; ++j) {
            double v = world.data[i*world.longitude + j];
            local_min = std::min(local_min, v);
            local_max = std::max(local_max, v);
            local_sum += v;
        }
    double global_min, global_max, global_sum;
    MPI_Allreduce(&local_min, &global_min, 1, MPLDOUBLE, MPLMIN, MPLCOMM_WORLD);
    MPI_Allreduce(&local_max, &global_max, 1, MPLDOUBLE, MPLMAX, MPLCOMM_WORLD);
    MPI_Allreduce(&local_sum, &global_sum, 1, MPLDOUBLE, MPLSUM, MPLCOMM_WORLD);
    double meant = global_sum / (world.global_latitude * world.global_longitude);
    if (mpi_rank == 0) {
        std::cout << "min:-" << global_min
                  << ", -max:-" << global_max

```

```

        << ",-avg:-" << meant << std::endl;
    }
}

/** Exchange the ghost cells i.e. copy the second data row and column to the very last da
 *
 * @param world The world to fix the boundaries for.
 */
void exchange_ghost_cells(World &world) {
    uint64_t lat = world.latitude;
    uint64_t lon = world.longitude;

    int rank_x = mpi_rank % mpi_size_x;
    int rank_y = mpi_rank / mpi_size_x;

    int left    = rank_y * mpi_size_x + (rank_x - 1 + mpi_size_x) % mpi_size_x;
    int right   = rank_y * mpi_size_x + (rank_x + 1) % mpi_size_x;
    int bottom  = ((rank_y - 1 + mpi_size_y) % mpi_size_y) * mpi_size_x + rank_x;
    int top     = ((rank_y + 1) % mpi_size_y) * mpi_size_x + rank_x;

    // — x direction (columns, non-contiguous) —
    if (mpi_size_x == 1) {
        for (uint64_t i = 0; i < lat; ++i) {
            world.data[i*lon + 0] = world.data[i*lon + lon - 2];
            world.data[i*lon + lon - 1] = world.data[i*lon + 1];
        }
    } else {
        std::vector<double> send_left(lat), send_right(lat);
        std::vector<double> recv_left(lat), recv_right(lat);
        for (uint64_t i = 0; i < lat; ++i) {
            send_left[i] = world.data[i*lon + 1];
            send_right[i] = world.data[i*lon + lon - 2];
        }
        //auto begin1 = std::chrono::steady_clock::now();
        MPI_Sendrecv(send_right.data(), (int)lat, MPI_DOUBLE, right, 0,
                     recv_left.data(), (int)lat, MPI_DOUBLE, left, 0,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(send_left.data(), (int)lat, MPI_DOUBLE, left, 1,
                     recv_right.data(), (int)lat, MPI_DOUBLE, right, 1,
                     MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        //auto end1 = std::chrono::steady_clock::now();
        //std::cout << "rank: " << mpi_rank << "time x: send-recv: " << ((end1 - begin1).
        for (uint64_t i = 0; i < lat; ++i) {
            world.data[i*lon + 0] = recv_left[i];
            world.data[i*lon + lon - 1] = recv_right[i];
        }
    }
}

// — y direction (rows, contiguous) —
if (mpi_size_y == 1) {
    for (uint64_t j = 0; j < lon; ++j) {
        world.data[0*lon + j] = world.data[(lat - 2)*lon + j];
        world.data[(lat - 1)*lon + j] = world.data[1*lon + j];
    }
}

```

```

    } else {
        //auto begin2 = std::chrono::steady_clock::now();
        MPI_Sendrecv(&world.data[(lat - 2)*lon], (int)lon, MPLDOUBLE, top,
2,
                    &world.data[0], (int)lon, MPLDOUBLE, bottom, 2,
                    MPLCOMM_WORLD, MPI_STATUS_IGNORE);
        MPI_Sendrecv(&world.data[1*lon], (int)lon, MPLDOUBLE, bottom, 3,
                    &world.data[(lat - 1)*lon], (int)lon, MPLDOUBLE, top,
3,
                    MPLCOMM_WORLD, MPI_STATUS_IGNORE);
        //auto end2 = std::chrono::steady_clock::now();
        //std::cout << "rank: " << mpi_rank << "time y: send-recv: " << ((end2 - begin2).
    }
}

/** Warm the world based on the position of the sun.
 *
 * @param world The world to warm.
 */
void radiation(World& world) {
    double sun_angle = std::cos(world.time);
    double sun_intensity = 865.0;
    double sun_long = (std::sin(sun_angle) * (world.global_longitude / 2))
        + world.global_longitude / 2.;
    double sun_lat = world.global_latitude / 2.;
    double sun_height = 100. + std::cos(sun_angle) * 100.;
    double sun_height_squared = sun_height * sun_height;

    for (uint64_t i = 1; i < world.latitude - 1; ++i) {
        for (uint64_t j = 1; j < world.longitude - 1; ++j) {
            // Euclidean distance between the sun and each earth coordinate
            double delta_lat = sun_lat - (i + world.offset_latitude);
            double delta_long = sun_long - (j + world.offset_longitude);
            double dist = sqrt(delta_lat*delta_lat +
                                delta_long*delta_long +
                                sun_height_squared);
            world.data[i * world.longitude + j] += \
                (sun_intensity / dist) * (1. - world.albedo_data[i * world.longitude + j])
        }
    }
    exchange_ghost_cells(world);
}

/** Heat radiated to space
 *
 * @param world The world to update.
 */
void energy_emmission(World& world) {
    for (uint64_t i = 0; i < world.latitude * world.longitude; ++i) {
        world.data[i] *= 0.99;
    }
}

/** Heat diffusion

```

```

*
* @param world The world to update.
*/
void diffuse(World& world) {
    std::vector<double> tmp = world.data;
    for (uint64_t k = 0; k < 10; ++k) {
        for (uint64_t i = 1; i < world.latitude - 1; ++i) {
            for (uint64_t j = 1; j < world.longitude - 1; ++j) {
                // 5 point stencil
                double center = world.data[i * world.longitude + j];
                double left = world.data[(i - 1) * world.longitude + j];
                double right = world.data[(i + 1) * world.longitude + j];
                double up = world.data[i * world.longitude + (j - 1)];
                double down = world.data[i * world.longitude + (j + 1)];
                tmp[i * world.longitude + j] = (center + left + right + up + down) / 5.;
            }
        }
        std::swap(world.data, tmp); // swap pointers for the two arrays
        exchange_ghost_cells(world); // update ghost zones
    }
}

/** One integration step at 'world_time'
*
* @param world The world to update.
*/
void integrate(World& world) {
    radiation(world);
    energy_emmission(world);
    diffuse(world);
}

/** Read a world model from a HDF5 file
*
* @param filename The path to the HDF5 file.
* @return A new world based on the HDF5 file.
*/
World read_world_model(const std::string& filename) {
    H5::H5File file(filename, H5F_ACC_RDONLY);
    H5::DataSet dataset = file.openDataSet("world");
    H5::DataSpace dataspace = dataset.getSpace();

    if (dataspace.getSimpleExtentNdims() != 2) {
        throw std::invalid_argument("Error while reading the model: the number of dimensions is not 2");
    }

    if (dataset.getTypeClass() != H5T_FLOAT or dataset.getFloatType().getSize() != 8) {
        throw std::invalid_argument("Error while reading the model: wrong data type, must be float64");
    }

    hsize_t dims[2];
    dataspace.getSimpleExtentDims(dims, NULL);
    std::vector<double> data_out(dims[0] * dims[1]);
    dataset.read(data_out.data(), H5::PredType::NATIVE_DOUBLE, dataspace, dataspace);
}

```

```

std::cout << "World-model-loaded---latitude:-" << (unsigned long) (dims[0]) << ",-lo
    << (unsigned long) (dims[1]) << std::endl;
return World(static_cast<uint64_t>(dims[0]), static_cast<uint64_t>(dims[1]), 293.15,
}

/** Write data to a hdf5 file
 *
 * @param group The hdf5 group to write in
 * @param name The name of the data
 * @param shape The shape of the data
 * @param data The data
 */
void write_hdf5_data(H5::Group& group, const std::string& name,
                    const std::vector<hsize_t>& shape, const std::vector<double>& data)
{
    H5::DataSpace dataspace(static_cast<int>(shape.size()), &shape[0]);
    H5::DataSet dataset = group.createDataSet(name.c_str(), H5::PredType::NATIVE_DOUBLE,
    dataset.write(&data[0], H5::PredType::NATIVE_DOUBLE);
}

/** Write a history of the world temperatures to a HDF5 file
 *S
 * @param world world to write
 * @param filename The output filename of the HDF5 file
 */
void write_hdf5(const World& world, const std::string& filename, uint64_t iteration) {
    static H5::H5File file(filename, H5F_ACC_TRUNC);

    H5::Group group(file.createGroup("/") + std::to_string(iteration));
    write_hdf5_data(group, "world", {world.latitude, world.longitude}, world.data);
}

/** Simulation of a flat world climate
 *
 * @param num_of_iterations Number of time steps to simulate
 * @param model_filename The filename of the world model to use (HDF5 file)
 * @param output_filename The filename of the written world history (HDF5 file)
 */
void simulate(uint64_t num_of_iterations, const std::string& model_filename, const std::s

    // Rank 0 reads the global model and broadcasts dimensions
    int global_dims[2] = {0, 0};
    std::vector<double> global_albedo;
    std::vector<double> global_data;

    if (mpi_rank == 0) {
        World global_world = read_world_model(model_filename);
        global_dims[0] = (int)global_world.latitude;
        global_dims[1] = (int)global_world.longitude;
        global_albedo = std::move(global_world.albedo_data);
    }

    MPI_Bcast(global_dims, 2, MPI_INT, 0, MPLCOMM_WORLD);
    uint64_t gl_latitude = global_dims[0];

```



```

uint64_t gl_longitude = global_dims[1];

// Compute 2D decomposition: find (sx, sy) with sx*sy = mpi_size that minimizes perim
// No even-divisibility requirement: remainder rows/columns are distributed across ranks
{
    int best_perim = -1;
    for (int sx = 1; sx <= mpi_size; ++sx) {
        if (mpi_size % sx != 0) continue;
        int sy = mpi_size / sx;
        // Use ceiling division for perimeter estimate
        int perim = (int)((gl_longitude + sx - 1) / sx) + ((gl_latitude + sy - 1) / sy);
        if (best_perim < 0 || perim < best_perim) {
            best_perim = perim;
            mpi_size_x = sx;
            mpi_size_y = sy;
        }
    }
}

int rank_x = mpi_rank % mpi_size_x;
int rank_y = mpi_rank / mpi_size_x;

// Each rank may have a different tile size (handles non-evenly-divisible grids)
uint64_t unit_lon = tile_size_for(gl_longitude, mpi_size_x, rank_x);
uint64_t unit_lat = tile_size_for(gl_latitude, mpi_size_y, rank_y);
uint64_t off_lon = tile_offset_for(gl_longitude, mpi_size_x, rank_x);
uint64_t off_lat = tile_offset_for(gl_latitude, mpi_size_y, rank_y);

// Local dimensions including ghost cells
const uint64_t longitude = unit_lon + 2;
const uint64_t latitude = unit_lat + 2;

// Offsets: -1 because cell index 0 is a ghost cell
const long int offset_longitude = (long int)off_lon - 1;
const long int offset_latitude = (long int)off_lat - 1;

// Distribute albedo data: broadcast full array, each rank extracts its local portion
// This avoids complex MPI_Scatterv packing for variable-sized tiles.
if (mpi_rank != 0) global_albedo.resize(gl_latitude * gl_longitude);
MPI_Bcast(global_albedo.data(), (int)(gl_latitude * gl_longitude), MPI_DOUBLE, 0, MPI_COMM_WORLD);

std::vector<double> local_albedo(longitude * latitude, 0.0);
for (uint64_t i = 0; i < unit_lat; ++i)
    for (uint64_t j = 0; j < unit_lon; ++j)
        local_albedo[(i + 1) * longitude + (j + 1)] =
            global_albedo[(i + off_lat) * gl_longitude + (j + off_lon)];
global_albedo.clear();
global_albedo.shrink_to_fit();

// Create local world
World world(latitude, longitude, 293.15, std::move(local_albedo));
world.global_latitude = gl_latitude;
world.global_longitude = gl_longitude;
world.offset_latitude = offset_latitude;

```

```

world.offset_longitude = offset_longitude;

// Rank 0 allocates buffer for global gather when writing output
if (mpi_rank == 0 && !output_filename.empty()) {
    global_data.resize(gl_latitude * gl_longitude);
}

if (mpi_rank == 0) {
    std::cout << "Decomposition:-" << mpi_size_x << "-x-" << mpi_size_y << std::endl;
}
std::cout << "rank-" << mpi_rank << "-owns-rows-"
    << offset_latitude + 1 << ",-" << offset_latitude + (long int)latitude - 2
    << "]" << "cols-"
    << offset_longitude + 1 << ",-" << offset_longitude + (long int)longitude -
    << "]" << "(tile:-" << unit_lon << "-x-" << unit_lat << ")" << std::endl;

exchange_ghost_cells(world);

// Pre-allocate reusable buffers for gather (avoids per-iteration allocation)
const uint64_t tile_size = unit_lat * unit_lon;
std::vector<double> local_interior(tile_size);

// Compute MPI_Gatherv counts and displacements (tiles may differ in size)
std::vector<int> recv_counts(mpi_size);
std::vector<int> recv_displs(mpi_size);
int total_gathered = 0;
for (int r = 0; r < mpi_size; ++r) {
    int rx = r % mpi_size_x;
    int ry = r / mpi_size_x;
    recv_counts[r] = (int)(tile_size_for(gl_longitude, mpi_size_x, rx)
        * tile_size_for(gl_latitude, mpi_size_y, ry));
    recv_displs[r] = total_gathered;
    total_gathered += recv_counts[r];
}

std::vector<double> all_tiles;
if (!output_filename.empty()) {
    if (mpi_rank == 0) all_tiles.resize(total_gathered);
}

// Pre-allocate the HDF5 output World once (only rank 0 uses it)
World* global_out_ptr = nullptr;
if (mpi_rank == 0 && !output_filename.empty()) {
    global_out_ptr = new World(gl_latitude, gl_longitude, 0.0,
        std::vector<double>(gl_latitude * gl_longitude, 0.0));
    global_out_ptr->global_latitude = gl_latitude;
    global_out_ptr->global_longitude = gl_longitude;
}

// set up counters and loop for num_iterations of integration steps
const double delta_time = world.global_longitude / 36.0;

auto begin = std::chrono::steady_clock::now();
for (uint64_t iteration = 0; iteration < num_of_iterations; ++iteration) {

```

```

world.time = iteration / delta_time;
integrate(world);

if (!output_filename.empty()) {
    // Pack interior data (strip ghost cells)
    for (uint64_t i = 0; i < unit_lat; ++i)
    for (uint64_t j = 0; j < unit_lon; ++j)
        local_interior[i * unit_lon + j] = world.data[(i + 1) * longitude + (j + 1) * longitude];

    // Gather all tiles on rank 0 using MPI_Gatherv (variable tile sizes)
    MPI_Gatherv(local_interior.data(), (int)tile_size, MPLDOUBLE,
                all_tiles.data(), recv_counts.data(), recv_displs.data(), MPLDOUBLE,
                0, MPLCOMM_WORLD);

    if (mpi_rank == 0) {
        // Unpack tiles from rank-order into 2D global array
        for (int r = 0; r < mpi_size; ++r) {
            int rx = r % mpi_size_x;
            int ry = r / mpi_size_x;
            uint64_t r_unit_lon = tile_size_for(gl_longitude, mpi_size_x, rx);
            uint64_t r_unit_lat = tile_size_for(gl_latitude, mpi_size_y, ry);
            uint64_t r_off_lon = tile_offset_for(gl_longitude, mpi_size_x, rx);
            uint64_t r_off_lat = tile_offset_for(gl_latitude, mpi_size_y, ry);
            const double* tile = all_tiles.data() + recv_displs[r];
            for (uint64_t i = 0; i < r_unit_lat; ++i)
            for (uint64_t j = 0; j < r_unit_lon; ++j)
                global_data[(i + r_off_lat) * gl_longitude + (j + r_off_lon)] =
                    tile[i * r_unit_lon + j];
        }

        // Write assembled global data to HDF5 (reuse pre-allocated World)
        global_out_ptr->data = global_data;
        write_hdf5(*global_out_ptr, output_filename, iteration);
    }

    // stat uses MPI_Allreduce — all ranks must call it
    if (mpi_rank == 0) std::cout << iteration << " — ";
    stat(world);
}
}
auto end = std::chrono::steady_clock::now();
delete global_out_ptr;

stat(world);
double cs = checksum(world);
if (mpi_rank == 0) {
    std::cout << "checksum-----" << cs << std::endl;
    std::cout << "elapsed-time--:" << (end - begin).count() / 1000000000.0 << "-sec"
    std::cout << (end - begin).count() / 1000000000.0 << std::endl;
}
}

/** Main function that parses the command line and start the simulation */
int main(int argc, char **argv) {

```

```

// Initialize the MPI environment
MPI_Init(&argc , &argv);

// Get the number of processes
MPI_Comm_size(MPLCOMM_WORLD, &mpi_size);

// Get the rank of the process
MPI_Comm_rank(MPLCOMM_WORLD, &mpi_rank);

// Get the name of the processor
char processor_name[MPLMAX_PROCESSOR_NAME];
int name_len;
MPI_Get_processor_name(processor_name , &name_len);

// Print off a hello world message
std::cout << mpi_size << std::endl;
std::cout << "Flat-World-Climate-running-on-" << processor_name
    << ",-rank-" << mpi_rank << "-out-of-" << mpi_size << std::endl;

uint64_t iterations=0;
std::string model_filename;
std::string output_filename;

std::vector <std::string> argument({argv , argv+argc});

for (long unsigned int i = 1; i<argument.size() ; i += 2){
    std::string arg = argument[i];
    if(arg=="-h"){ // Write help
        std::cout << " ./fwc --iter-<number-of-iterations>"
            << " --model-<input-model>"
            << " --out-<name-of-output-file>\n";
        exit(0);
    } else if (i == argument.size() - 1)
        throw std::invalid_argument("The last argument-(" + arg + ")-must-have-a-value");
    else if(arg=="--iter"){
        if ((iterations = std::stoi(argument[i+1])) < 0)
            throw std::invalid_argument("iter-most-be-positive-(e.g.--iter-1000)");
    } else if(arg=="--model"){
        model_filename = argument[i+1];
    } else if(arg=="--out"){
        output_filename = argument[i+1];
    } else{
        std::cout << "-->-error:-the-argument-type-is-not-recognized-\n";
    }
}
if (model_filename.empty())
    throw std::invalid_argument("You-must-specify-the-model-to-simulate-"
        " (e.g. --model-models/small.hdf5)");
if (iterations==0)
    throw std::invalid_argument("You-must-specify-the-number-of-iterations-"
        " (e.g. --iter-10)");
std::cout << "rank:-" << mpi_rank << " --size:-" << mpi_size << std::endl;

```

```
    simulate(iterations , model_filename , output_filename );  
    MPI_Finalize ();  
    return 0;  
}
```